PHP form

The following *index.php* contains a form that allows users to subscribe to a newsletter:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="stylesheet" href="css/style.css">
    <title>PHP Form Demo</title>
</head>
<body>
    <main>
        <form action="subscribe.php" method="post">
            <div>
                <label for="name">Name:</label>
                <input type="text" name="name" required="required" placeholder="Enter your name" />
            </div>

            <div>
                <label for="name">Email:</label>
                <input type="email" name="email" required="required" placeholder="Enter your email" />
            </div>

            <button type="submit">Subscribe</button>
        </form>
    </main>
</body>
```

```
</html>
```

- The form uses the *POST* method and submits the data to the *subscribe.php* page. It has two input elements with type *text* and *email*.
- Before submitting the form to the server, the web browser checks if all required fields are filled out, in the correct format. This is known as client-side form validation.
- To validate data on the client-side, you can use either HTML5 form validation or JavaScript validation. However, you should never rely on client-side validation as a security measure.
- The client-side validation's purpose is to help legitimate users(means a user of a Customer Product that (i) acquired the Customer Product lawfully, and (ii) uses the Embedded Software embedded on the Customer Product strictly limited to the extent reasonably necessary for using the Customer Product and in line with applicable copyright laws.) to enter data in the correct format. However, it doesn't prevent malicious users from entering bad data that potentially harm the application. Therefore, it's mandatory to implement server-side validation.

The following shows the code of the *subscribe.php* file:

```
<?php


//-----------------------------------------
// WARNING: this doesn't include sanitization
// and validation
//-----------------------------------------
if (isset($_POST['name'], $_POST['email'])) {
        $name = $_POST['name'];
        $email = $_POST['email'];


        // show the $name and $email
        echo "Thanks $name for your subscription.<br>";
        echo "Please confirm it in your inbox of the email $email.";
} else {
        echo 'You need to provide your name and email address.';
}
```

How it works.

- First, check if the name and email are in the *$_POST* array using the *isset()* function.
- Second, show a thank you message.

If you enter the name as *John* and email as *john.doe@example.com*, you'll see the following message on the *subscribe.php* page:

Thanks John for your subscription.

Please confirm it in your inbox of the email john.doe@example.com

Escaping the output

- The *subscribe.php* page directly displays the form data. If malicious hackers intentionally enter bad data, the page won't work properly.
- For example, if the following JavaScript code is entered in the *name* field and the form is submitted.

```
<script>alert('Hello');</script>
```

…you'll see that the page displays an alert.

Imagine that the script doesn't just show an alert but loads the malicious code from another server to the user's web browser, the risk is higher. This type of attack is called cross-site scripting (XSS) attack.

Therefore, before displaying user input on a webpage, you should always escape the data. To do that, you use the *htmlspecialchars()* function:

```
<?php


//-----------------------------------------
// WARNING: this doesn't include sanitization
// and validation
//-----------------------------------------
if (isset($_POST['name'], $_POST['email'])) {
        $name = htmlspecialchars($_POST['name']);
        $email = htmlspecialchars($_POST['email']);


        // show the $name and $email
        echo "Thanks $name for your subscription.<br>";
```

```
        echo "Please confirm it in your inbox of the email $email.";
} else {
        echo 'You need to provide your name and email address.';
}
```

PHP Get Form

Get request is the default form request. The data passed through get request is visible on the URL browser so it is not secured. You can send limited amount of data through get request.

Let's see a simple example to receive data from get request in PHP.

File: form1.php

```
1. <form action="welcome.php" method="get">
2. Name: <input type="text" name="name"/>
3. <input type="submit" value="visit"/>
4. </form>
```

File: welcome.php

```
1. <?php
2. $name=$_GET["name"];//receiving name field value in $name variable
3. echo "Welcome, $name";
4. ?>
```

PHP Post Form

Post request is widely used to submit form that have large amount of data such as file upload, image upload, login form, registration form etc.
The data passed through post request is not visible on the URL browser so it is secured. You can send large amount of data through post request.
Let's see a simple example to receive data from post request in PHP.
File: form1.php

```
1. <form action="login.php" method="post">
2. <table>
3. <tr><td>Name:</td><td> <input type="text" name="name"/></td></tr>
4. <tr><td>Password:</td><td> <input type="password" name="password"/></td></tr>
5. <tr><td colspan="2"><input type="submit" value="login"/>  </td></tr>
6. </table>
```

```
7. </form>
```

File: login.php

```
1. <?php
2. $name=$_POST["name"];//receiving name field value in $name variable
3. $password=$_POST["password"];//receiving password field value in $password variable
4.
5. echo "Welcome: $name, your password is: $password";
6. ?>
```

Building forms in php[without self-processing pages]

Building forms in PHP involves creating the HTML markup for the form and using PHP to handle form submissions. Below is a step-by-step guide:

1. Create the HTML Form:

Start by creating an HTML form with the necessary input fields. Save this HTML code in a file, for example, index.php.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>PHP Form Example</title>
</head>
<body>

  <h2>Contact Us</h2>

  <form action="process_form.php" method="post">
    <label for="name">Name:</label>
    <input type="text" name="name" id="name" required>

    <br>

    <label for="email">Email:</label>
    <input type="email" name="email" id="email" required>

    <br>

    <label for="message">Message:</label>
    <textarea name="message" id="message" rows="4" required></textarea>
```

```
    <br>

    <input type="submit" value="Submit">
  </form>

</body>
</html>
```

© In this example, the form sends data to a file named process_form.php using the post method.

2. Create the PHP Script to Process Form Submissions (process_form.php):

Create a new PHP file named process_form.php to handle form submissions.

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
  // Retrieve form data
  $name = $_POST["name"];
  $email = $_POST["email"];
  $message = $_POST["message"];

  // Do something with the data (e.g., send an email, save to a database)

  // For demonstration purposes, just echoing the data
  echo "Name: $name<br>";
  echo "Email: $email<br>";
  echo "Message: $message";
} else {
  // Handle the case where the form is not submitted properly
  echo "Form not submitted!";
}
?>
```

In this script:

- It checks if the form has been submitted using $_SERVER["REQUEST_METHOD"].
- It retrieves the form data using $_POST.
- You can replace the comment with the actual code to process the form data (e.g., send an email, save to a database).

3. Run Your Application:

- Save both files in the same directory and run index.php in a web browser. Fill out the form and submit it. The form data should be processed by process_form.php.

- Remember to validate and sanitize user input to prevent security vulnerabilities like SQL injection or cross-site scripting (XSS). Use functions like filter_ var() or prepared statements for database interactions.

how to use $_GET

In PHP, $_GET is an associative array that contains data sent to the server via URL parameters. When a form is submitted using the GET method or when parameters are included in the URL, PHP can access this data through the $_GET array. Here's a simple example to illustrate how to **use $_GET:**

1. **Create an HTML Form (index.php):**

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>GET Form Example</title>
</head>
<body>

   <h2>GET Form</h2>

   <form action="process_form.php" method="get">
      <label for="name">Name:</label>
      <input type="text" name="name" id="name" required>

      <br>

      <label for="email">Email:</label>
      <input type="email" name="email" id="email" required>

      <br>

      <input type="submit" value="Submit">
   </form>

</body>
</html>
```

2. **Create the PHP Script to Process Form Submissions (process_form.php):**

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "GET") {
   // Retrieve form data using $_GET
   $name = isset($_GET["name"]) ? $_GET["name"] : "";
   $email = isset($_GET["email"]) ? $_GET["email"] : "";

   // Display the received data
   echo "Name: $name<br>";
   echo "Email: $email";
} else {
   echo "Form not submitted using GET method!";
}
```

```
?>
```

In this script:

- It checks if the form has been submitted using the GET method using $_SERVER["REQUEST_METHOD"].
- It retrieves the form data using $_GET.
- It displays the received data.

3. Run Your Application:

- Save both files in the same directory and run index.php in a web browser. Fill out the form and submit it. The form data should be processed by process_form.php and displayed.

- Keep in mind that using $_GET makes data visible in the URL, which may not be suitable for sensitive information. It's more commonly used for search queries and other scenarios where the data's visibility is not a concern. If you're dealing with sensitive data, consider using the POST method instead.

- Remember to validate and sanitize user input to prevent security vulnerabilities. Use functions like filter_var() to validate and escape user input when necessary.

htmlspecialchars() in HTTP_GET method and HTTP_POST method

© When using user input in an HTML document that is generated based on data from the HTTP GET or POST method, it's a good practice to use the htmlspecialchars() function to prevent Cross-Site Scripting (XSS) attacks. This is important for both GET and POST requests.

Using htmlspecialchars() with HTTP GET Method:

© When processing data received through the GET method, you should apply htmlspecialchars() to any user input before displaying it on a web page.

Example:

```
<?php
// Retrieve data from GET request
$name = isset($_GET['name']) ? $_GET['name'] : '';

// Use htmlspecialchars to prevent XSS
$escapedName = htmlspecialchars($name, ENT_QUOTES, 'UTF-8');
?>

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTTP GET Example</title>
```

```
</head>
<body>

   <h2>Welcome, <?php echo $escapedName; ?></h2>

</body>
</html>
```

Using htmlspecialchars() with HTTP POST Method:

Similarly, when processing data received through the POST method, apply htmlspecialchars() to user input before displaying it.

Example:

```
<?php
// Retrieve data from POST request
$name = isset($_POST['name']) ? $_POST['name'] : '';

// Use htmlspecialchars to prevent XSS
$escapedName = htmlspecialchars($name, ENT_QUOTES, 'UTF-8');
?>

<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>HTTP POST Example</title>
</head>
<body>

   <h2>Welcome, <?php echo $escapedName; ?></h2>

</body>
</html>
```

- © In both examples, the htmlspecialchars() function is used to encode special characters in the user input, making it safe to output in HTML. The ENT_QUOTES flag is used to ensure that both single and double quotes are encoded.

- © Always validate and sanitize user input before using it, and use htmlspecialchars() when rendering the data in HTML to prevent potential security vulnerabilities.

Form Parameters

Use the $_POST, $_GET, and $_FILES arrays to access form parameters from your PHP code. The keys are the parameter names, and the values are the values of those parameters. Because

periods are legal in HTML field names but not in PHP variable names, periods in field names are converted to underscores (_) in the array.

A complete PHP program that includes an HTML form for uploading files using the POST method. The corresponding PHP script (process_upload.php) handles the uploaded file.

File Uploads

To handle file uploads (supported in most modern browsers), use the $_FILES array. Using the various authentication and file upload functions, you can control who is allowed to upload files and what to do with those files once they're on your system.

index_upload.php (HTML form with file upload):

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>PHP File Upload Example</title>
</head>
<body>

   <h2>File Upload Form</h2>

   <form action="process_upload.php" method="post" enctype="multipart/form-data">
      <label for="file">Select a file:</label>
      <input type="file" name="file" id="file" required>

      <br>

      <input type="submit" value="Upload">
   </form>

</body>
</html>
```

process_upload.php (PHP script to process file upload):

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
   // Check if file was uploaded without errors
   if (isset($_FILES["file"]) && $_FILES["file"]["error"] == 0) {
      $file_name = $_FILES["file"]["name"];
      $file_size = $_FILES["file"]["size"];
      $file_tmp = $_FILES["file"]["tmp_name"];
      $file_type = $_FILES["file"]["type"];

      // Move the uploaded file to a designated directory
```

```
        $upload_directory = "uploads/";
        $target_path = $upload_directory . basename($file_name);

        if (move_uploaded_file($file_tmp, $target_path)) {
            echo "File uploaded successfully:<br>";
            echo "File Name: $file_name<br>";
            echo "File Size: $file_size bytes<br>";
            echo "File Type: $file_type";
        } else {
            echo "Error uploading the file.";
        }
    } else {
        echo "Error: " . $_FILES["file"]["error"];
    }
} else {
    // Handle the case where the form is not submitted properly
    echo "Form not submitted using POST method!";
}
?>
```

In this example:

- The HTML form has an <input type="file"> element to allow users to select a file for upload.
- The form has enctype="multipart/form-data" to support file uploads.
- The PHP script checks if the file was uploaded successfully and moves it to a designated directory (uploads/ in this case).
- The script then displays information about the uploaded file.
- To use this program, create a directory on your web server, place these files in the directory, and ensure that the uploads/ directory has write permissions.

- Visit http://yourdomain/index_upload.php in your web browser to access the file upload form. After selecting a file and submitting the form, the uploaded file information will be displayed on the page.

Self-Processing Pages

- Self-processing pages in PHP are web pages that process user data and display the results without requiring the user to submit the data to another page. The processing is done on the same page where the form is located, using PHP script. The script retrieves the data entered by the user, performs any necessary calculations or data manipulation, and displays the results to the user. To create a self-processing page in PHP, the page must contain an HTML form and a PHP script that processes the data from the form.
- Sometimes, you want to include both form and logic for handling form submission in a single PHP file. This form is often referred to as a self-processing form.

- To create a self-processing form, you can use the *$_SERVER['REQUEST_METHOD']* that returns the request method e.g., *GET* or *POST*.

- © If the *$_SERVER['REQUEST_METHOD']* is GET, you show the form. And if the *$_SERVER['REQUEST_METHOD']* is POST, you process it.

You should validate and sanitize the form data to prevent security vulnerabilities such as SQL injection and XSS attacks.

- © For example, if the executing script is *https://www.phptutorial.net/app/form/index.php*, the *$_SERVER['PHP_SELF']* returns */app/form/index.php*.
- © However, the *$_SERVER['PHP_SELF']* cannot be trusted since and it's vulnerable to XSS attacks. Therefore, you should always escape it using the *htmlspecialchars()* function.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="css/style.css">
    <title>PHP Form</title>
</head>
<body>
    <main>
        <?php if ($_SERVER['REQUEST_METHOD'] === 'GET') : ?>
            <form action="<?php htmlspecialchars($_SERVER['PHP_SELF']) ?>"
method="post">
                <div>
                    <label for="name">Name:</label>
                    <input type="text" name="name" required="required"
placeholder="Enter your name" />
                </div>

                <div>
                    <label for="name">Email:</label>
                    <input type="email" name="email" required="required"
placeholder="Enter your email" />
                </div>

                <button type="submit">Subscribe</button>

            </form>
        <?php else : ?>

            <?php
            //---------------------------------------------
            // WARNING: this doesn't include sanitization
```

```
            // and validation
            //-------------------------------------------
            if (isset($_POST['name'], $_POST['email'])) {
                $name = htmlspecialchars($_POST['name']);
                $email = htmlspecialchars($_POST['email']);

                // show the $name and $email
                echo "Thanks $name for your subscription.<br>";
                echo "Please confirm it in your inbox of the email $email.";
            } else {
                echo 'You need to provide your name and email address.';
            }

            ?>

        <?php endif ?>
    </main>
</body>

</html>
```

Example 2:

```
<html>
<head><title>Temperature Conversion</title></head>
<body>
<?php if ($_SERVER['REQUEST_METHOD'] == 'GET') { ?>
<form action="<?php echo $_SERVER['PHP_SELF'] ?>" method="POST">
 Fahrenheit temperature:
<input type="text" name="fahrenheit" /><br />
<input type="submit" value="Convert to Celsius!" />
</form>
<?php }
else if ($_SERVER['REQUEST_METHOD'] == 'POST') {
$fahrenheit = $_POST['fahrenheit'];
$celsius = ($fahrenheit - 32) * 5 / 9;
printf("%.2fF is %.2fC", $fahrenheit, $celsius);
}
else {
die("This script only works with GET and POST requests.");

}
//Example: A self-processing temperature conversion page
 ?>
</body>
</html>
```

Processing Forms in php

Processing Forms

It's easy to process forms with PHP, as the form parameters are available in the $_GET and $_POST arrays. This section describes some tricks and techniques that will make it even easier.

Index1.php

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Contact Form</title>
</head>
<body>
  <form action="form-handler.php" method="post">
    <label for="stu_name">Name:</label>
    <input type="text" id="stu_name" name="stu_name" required>

    <label for="lastname">Lastname:</label>
    <input type="text" id="lastname" name="lastname" required>

    <input type="submit" value="Submit">
  </form>
</body>
</html>
```

form-handler.php

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
  // Retrieve form data
  $stu_name = $_POST['stu_name'];
  $lastname = $_POST['lastname'];

  // Perform validation and sanitization on the form data
  $stu_name = htmlspecialchars($stu_name);
  $lastname = htmlspecialchars($lastname);

  // Insert the form data into the database or send an email, etc.
  // Example: Assuming you have a database connection

  $servername = "localhost";
$username = "root";
$password = "";
$dbname = "subash_db";

  // Create connection
  $conn = new mysqli($servername, $username, $password, $dbname);
```

```
  // Check connection
  if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
  }
  // Insert data into the database
  $sql = "INSERT INTO pkstudent (stu_name,lastname) VALUES ('$stu_name','$lastname')";

  if ($conn->query($sql) === TRUE) {
    $_SESSION['success_message'] = "New record created successfully";
  } else {
    echo "Error: " . $sql . "<br>" . $conn->error;
  }
  $conn->close();
  // Redirect the user to a thank you page
  header('Location: thanks.php');
  exit;
}
```

Thanks.php

```
?>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Thank You</title>
</head>
<body>
  <h1>Thank You!</h1>
  <?php
  // Check if the success message is set
  if (isset($_SESSION['success_message'])) {
    echo '<p>' . $_SESSION['success_message'] . '</p>';
    // Unset the session variable to prevent displaying the message on subsequent visits
    unset($_SESSION['success_message']);
  }
  ?>
  <p>Your form submission has been received. We appreciate your input.</p>
  <!-- Additional content or links can be added here -->
</body>
</html>
```

Sticky Forms in php

- Sticky forms are a common technique used in web development to preserve user input in form fields after a form submission.

- This is especially helpful when there are validation errors, and you want to display the form again with the previously entered values so that the user doesn't have to re-enter everything.

Here's a simple example of how you can create a sticky form in PHP:

```php
<?php
// Initialize variables to store form field values
$name = '';
$email = '';

// Check if the form has been submitted
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
   // Retrieve form data
   $name = $_POST['name'];
   $email = $_POST['email'];

   // Validate form data (you can add more validation as needed)
   $errors = array();

   if (empty($name)) {
      $errors[] = 'Name is required';
   }

   if (empty($email)) {
      $errors[] = 'Email is required';
   }

   // If there are no validation errors, you can proceed with further processing
   if (empty($errors)) {
      // Your form processing logic goes here

      // Redirect or display a success message
      header('Location: success.php');
      exit;
   }
}

?>

<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Sticky Form Example</title>
</head>
<body>

<h1>Contact Form</h1>
```

```
<?php
// Display validation errors, if any
if (!empty($errors)) {
   echo '<div style="color: red;">';
   echo '<ul>';
   foreach ($errors as $error) {
      echo '<li>' . $error . '</li>';
   }
   echo '</ul>';
   echo '</div>';
}
?>

<form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>">
   <label for="name">Name:</label>
   <input type="text" name="name" id="name" value="<?php echo htmlspecialchars($name); ?>">
   <br>

   <label for="email">Email:</label>
   <input type="email" name="email" id="email" value="<?php echo htmlspecialchars($email);
?>">
   <br>

   <input type="submit" value="Submit">
</form>

</body>
</html>
```

In this example:

- The form fields ($name and $email) are initialized with empty values.
- After a form submission, the values are retrieved from $_POST.
- Validation is performed, and if there are errors, they are displayed above the form.
- The htmlspecialchars function is used to prevent XSS attacks by escaping HTML characters in user input when displaying them in the form.
- This is a basic example, and you can customize it according to your specific requirements. Additionally, you may want to consider more advanced validation, security measures, and other features based on your application needs.

success.php

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Success Page</title>
</head>
<body>
```

```
<h1>Thank you for submitting the form!</h1>

<p>We have received your information and will process it shortly.</p>

</body>
</html>
```

Example :2

```
<html>
<head>
<title>Temperature Conversion</title>
</head>
<body>
<?php
//Example:Temperature conversion using the GET method (temp2.php)
if (isset ( $_GET ['fahrenheit'] )) {
$fahrenheit = $_GET ['fahrenheit'];
} else {
$fahrenheit = null;
}

if (is_null ( $fahrenheit )) {
?>
<form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="GET">
 Fahrenheit temperature: <input type="text" name="fahrenheit" /><br />
<input type="submit" value="Convert to Celsius!" />
</form>
<?php
} else {
$celsius = ($fahrenheit - 32) * 5 / 9;
printf ( "%.2fF is %.2fC", $fahrenheit, $celsius );
}
//In Example: we copy the form parameter value into $fahrenheit. If we weren't
//given that parameter, $fahrenheit contains NULL, so we could use is_null() to
test
//whether we should display the form or process the form data.
?>
</body>
</html>
```

Multivalued Parameters in php

- In PHP, multivalued parameters refer to parameters that can have multiple values, typically when dealing with form submissions where multiple inputs share the same name.

- This is common when working with checkboxes, multiple select options, or arrays of form elements.

Here's an example to illustrate multivalued parameters:

1. HTML Form (example_form.php)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>MultiValued Parameters Example</title>
</head>
<body>

<form method="post" action="process_form.php">
   <label for="colors">Select your favorite colors:</label><br>
   <input type="checkbox" name="colors[]" value="red"> Red<br>
   <input type="checkbox" name="colors[]" value="blue"> Blue<br>
   <input type="checkbox" name="colors[]" value="green"> Green<br>

   <input type="submit" value="Submit">
</form>

</body>
</html>
```

In this example, the checkboxes share the same name (colors[]) and thus can have multiple selected values.

PHP Processing Script (process_form.php)

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
   // Check if 'colors' parameter is set in the $_POST array
   if (isset($_POST['colors'])) {
      // Retrieve the selected colors
      $selectedColors = $_POST['colors'];

      // Display the selected colors
      echo "<h2>Selected Colors:</h2>";
      echo "<ul>";
      foreach ($selectedColors as $color) {
         echo "<li>" . htmlspecialchars($color) . "</li>";
      }
      echo "</ul>";
   } else {
```

```
        echo "<p>No colors selected.</p>";
    }
} else {
    // If the form was not submitted via POST, redirect to the form page
    header('Location: example_form.php');
    exit;
}
?>
```

- In the processing script (process_form.php), it checks if the colors parameter is set in the $_POST array, which indicates that at least one checkbox has been selected. It then retrieves and displays the selected colors.

- Remember to handle user input carefully to prevent security issues, such as using htmlspecialchars to prevent HTML injection. Also, consider additional validation based on your specific requirements.

Form Validation

- **When you allow users to input data, you typically need to validate that data before using it or storing it for later use.**
- **There are several strategies available for validating data.**
- **The first is JavaScript on the client side.**
- **However, since the user can choose to turn JavaScript off, or may even be using a browser that doesn't support it, this cannot be the only validation you do.**

Form validation (data_validation.php)

```
<?php
// Function to sanitize input data
function sanitizeInput($data)
{
    $data = trim($data);
    $data = stripslashes($data);
    $data = htmlspecialchars($data);
    return $data;
}

// Function to validate the status
function isValidStatus($status)
{
    return ($status == "active" || $status == "inactive");
}

// Function to validate the media type
function isValidMediaType($type)
{
    $validTypes = ["picture", "audio", "movie"];
    return in_array($type, $validTypes);
```

```
}

// Initialize $tried
$tried = false;

// Initialize other variables
$name = $mediaType = $filename = $caption = $status = '';

// Process form submission
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $tried = true;

    // Sanitize form data
    $name = sanitizeInput($_POST['name']);
    $mediaType = sanitizeInput($_POST['media_type']);
    $filename = sanitizeInput($_POST['filename']);
    $caption = sanitizeInput($_POST['caption']);
    $status = isset($_POST['status']) ? sanitizeInput($_POST['status']) : '';

    // Validation
    $errors = [];
    if (empty($name)) {
        $errors[] = "Name is required.";
    }

    if (empty($mediaType) || !isValidMediaType($mediaType)) {
        $errors[] = "Invalid media type selected.";
    }

    if (empty($filename)) {
        $errors[] = "File name is required.";
    }

    if (empty($caption)) {
        $errors[] = "Caption is required.";
    } elseif (strlen($caption) > 255) { // Adjust the maximum length as needed
        $errors[] = "Caption should not exceed 255 characters.";
    }

    if (!isValidStatus($status)) {
        $errors[] = "Invalid status selected.";
    }

    // Display errors
    if (!empty($errors)) {
        echo "<ul>";
        foreach ($errors as $error) {
            echo "<li>$error</li>";
```

```
        }
        echo "</ul>";
    } else {
        echo "<p>The item has been created.</p>";
        // Additional processing or database insertion can be done here
    }
}
?>

<form action="<?php echo htmlspecialchars($_SERVER['PHP_SELF']); ?>" method="POST">
    Name: <input type="text" name="name" value="<?php echo $name; ?>" /><br />
    Status: <input type="checkbox" name="status" value="active" <?php echo isset($status) && $status
== "active" ? "checked" : ""; ?> /> Active<br />
    Media: <select name="media_type">
        <option value="">Choose one</option>
        <option value="picture" <?php echo $mediaType === "picture" ? "selected" : "";
?>>Picture</option>
        <option value="audio" <?php echo $mediaType === "audio" ? "selected" : ""; ?>>Audio</option>
        <option value="movie" <?php echo $mediaType === "movie" ? "selected" : ""; ?>>Movie</option>
    </select><br />
    File: <input type="text" name="filename" value="<?php echo $filename; ?>" /><br />
    Caption: <textarea name="caption"><?php echo $caption; ?></textarea><br />
    <input type="hidden" name="tried" value="yes" />
    <input type="submit" value="<?php echo $tried ? "Continue" : "Create"; ?>" />
</form>
```

1. Empty String
2. Validate String
3. Validate Numbers
4. Validate Email
5. Validate URL
6. Input length

Empty String

The code below checks that the field is not empty. If the user leaves the required field empty, it will show an error message. Put these lines of code to validate the required field.

```
if (emptyempty ($_POST["name"])) {
    $errMsg = "Error! You didn't enter the Name.";
            echo $errMsg;
} else {
    $name = $_POST["name"];
```

```
}
```

Validate String

The code below checks that the field will contain only alphabets and whitespace, for example - name. If the name field does not receive valid input from the user, then it will show an error message:

```
$name = $_POST ["Name"];
if (!preg_match ("/^[a-zA-z]*$/", $name) ) {
    $ErrMsg = "Only alphabets and whitespace are allowed.";
            echo $ErrMsg;
} else {
    echo $name;
}
```

Validate Number

The below code validates that the field will only contain a numeric value. For example - Mobile no. If the Mobile no field does not receive numeric data from the user, the code will display an error message:

```
$mobileno = $_POST ["Mobile_no"];
if (!preg_match ("/^[0-9]*$/", $mobileno) ){
    $ErrMsg = "Only numeric value is allowed.";
    echo $ErrMsg;
} else {
    echo $mobileno;
}
```

Validate Email

A valid email must contain @ and . symbols. PHP provides various methods to validate the email address. Here, we will use regular expressions to validate the email address.

The below code validates the email address provided by the user through HTML form. If the field does not contain a valid email address, then the code will display an error message:

```
$email = $_POST ["Email"];
$pattern = "^[_a-z0-9-]+(\.[_a-z0-9-]+)*@[a-z0-9-]+(\.[a-z0-9-]+)*(\.[a-z]{2,3})$^";
if (!preg_match ($pattern, $email) ){
```

```
    $ErrMsg = "Email is not valid.";
            echo $ErrMsg;
} else {
    echo "Your valid email address is: " .$email;
}
```

Input Length Validation

The input length validation restricts the user to provide the value between the specified range, for Example - Mobile Number. A valid mobile number must have 10 digits.

The given code will help you to apply the length validation on user input:

```
$mobileno = strlen ($_POST ["Mobile"]);
$length = strlen ($mobileno);

if ( $length < 10 && $length > 10) {
    $ErrMsg = "Mobile must have 10 digits.";
            echo $ErrMsg;
} else {
    echo "Your Mobile number is: " .$mobileno;
}
```

Validate URL

The below code validates the URL of website provided by the user via HTML form. If the field does not contain a valid URL, the code will display an error message, i.e., "URL is not valid".

```
$websiteURL = $_POST["website"];
if (!preg_match("/\b(?:(?:https?|ftp):\/\/|www\.)[-a-z0-9+&@#\/%?=~_|!:,.;]*[-a-z0-9+&@#\/%=~_|]/i",$website)) {
  $websiteErr = "URL is not valid";
  echo $websiteErr;
} else {
    echo "Website URL is: " .$websiteURL;
}
```

Button Click Validate

The below code validates that the user click on submit button and send the form data to the server one of the following method - get or post.

```
1. if (isset ($_POST['submit']) {
2.     echo "Submit button is clicked.";
3.     if ($_SERVER["REQUEST_METHOD"] == "POST") {
4.         echo "Data is sent using POST method ";
5.     }
6. } else {
7.     echo "Data is not submitted";
8. }
```

Note: Remember that validation and verification both are different from each other.

Now we will apply all these validations to an HTML form to validate the fields. Thereby you can learn in detail how these codes will be used to validation form.

Create a registration form using HTML and perform server-side validation. Follow the below instructions as given:

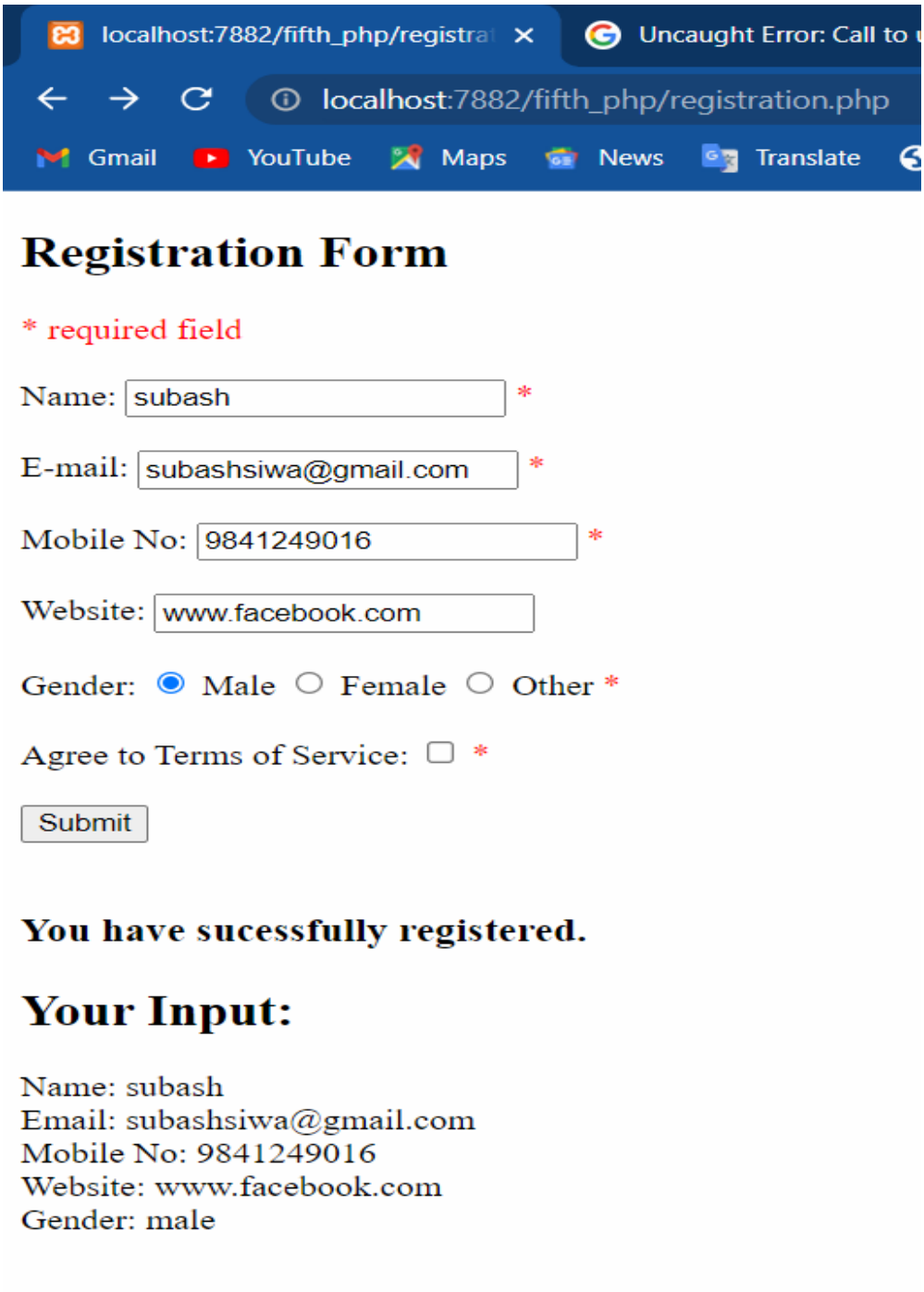
Create and validate a Registration form

```
<!DOCTYPE html>
<html>
<head>
<style>
.error {color: #FF0001;}
</style>
</head>
<body>

<?php
// define variables to empty values
$nameErr = $emailErr = $mobilenoErr = $genderErr = $websiteErr = $agreeErr = "";
$name = $email = $mobileno = $gender = $website = $agree = "";

//Input fields validation
if ($_SERVER["REQUEST_METHOD"] == "POST") {

//String Validation
   if (empty($_POST["name"])) {
       $nameErr = "Name is required";
   } else {
      $name = input_data($_POST["name"]);
          // check if name only contains letters and whitespace
```

```
        if (!preg_match("/^[a-zA-Z ]*$/",$name)) {
            $nameErr = "Only alphabets and white space are allowed";
        }
    }

    //Email Validation
    if (empty($_POST["email"])) {
        $emailErr = "Email is required";
    } else {
        $email = input_data($_POST["email"]);
        // check that the e-mail address is well-formed
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            $emailErr = "Invalid email format";
        }
     }

    //Number Validation
    if (empty($_POST["mobileno"])) {
        $mobilenoErr = "Mobile no is required";
    } else {
        $mobileno = input_data($_POST["mobileno"]);
        // check if mobile no is well-formed
        if (!preg_match ("/^[0-9]*$/", $mobileno) ) {
        $mobilenoErr = "Only numeric value is allowed.";
        }
      //check mobile no length should not be less and greator than 10
      if (strlen ($mobileno) != 10) {
        $mobilenoErr = "Mobile no must contain 10 digits.";
        }
    }

    //URL Validation
    if (empty($_POST["website"])) {
      $website = "";
    } else {
        $website = input_data($_POST["website"]);
        // check if URL address syntax is valid
        if (!preg_match("/\b(?:(?:https?|ftp):\/\/|www\.)[-a-z0-9+&@#\/%?=~_|!:,.;]*[-a-z0-
9+&@#\/%=~_|]/i",$website)) {
            $websiteErr = "Invalid URL";
        }
    }

    //Empty Field Validation
    if (empty ($_POST["gender"])) {
```

```
        $genderErr = "Gender is required";
    } else {
        $gender = input_data($_POST["gender"]);
    }

    //Checkbox Validation
    if (!isset($_POST['agree'])){
        $agreeErr = "Accept terms of services before submit.";
    } else {
        $agree = input_data($_POST["agree"]);
    }
}
function input_data($data) {
  $data = trim($data);
  $data = stripslashes($data);
  $data = htmlspecialchars($data);
  return $data;
}
?>

<h2>Registration Form</h2>
<span class = "error">* required field </span>
<br><br>
<form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>" >
    Name:
    <input type="text" name="name">
    <span class="error">* <?php echo $nameErr; ?> </span>
    <br><br>
    E-mail:
    <input type="text" name="email">
    <span class="error">* <?php echo $emailErr; ?> </span>
    <br><br>
    Mobile No:
    <input type="text" name="mobileno">
    <span class="error">* <?php echo $mobilenoErr; ?> </span>
    <br><br>
    Website:
    <input type="text" name="website">
    <span class="error"><?php echo $websiteErr; ?> </span>
    <br><br>
    Gender:
    <input type="radio" name="gender" value="male"> Male
    <input type="radio" name="gender" value="female"> Female
    <input type="radio" name="gender" value="other"> Other
    <span class="error">* <?php echo $genderErr; ?> </span>
```

```
   <br><br>
   Agree to Terms of Service:
   <input type="checkbox" name="agree">
   <span class="error">* <?php echo $agreeErr; ?> </span>
   <br><br>
   <input type="submit" name="submit" value="Submit">
   <br><br>
</form>

<?php
   if(isset($_POST['submit'])) {
   if($nameErr == "" && $emailErr == "" && $mobilenoErr == "" && $genderErr == "" && $websiteErr
== "" && $agreeErr == "") {
      echo "<h3 color = #FF0001> <b>You have sucessfully registered.</b> </h3>";
      echo "<h2>Your Input:</h2>";
      echo "Name: " .$name;
      echo "<br>";
      echo "Email: " .$email;
      echo "<br>";
      echo "Mobile No: " .$mobileno;
      echo "<br>";
      echo "Website: " .$website;
      echo "<br>";
      echo "Gender: " .$gender;
   } else {
      echo "<h3> <b>You didn't filled up the form correctly.</b> </h3>";
   }
   }
?>

</body>
</html>
```

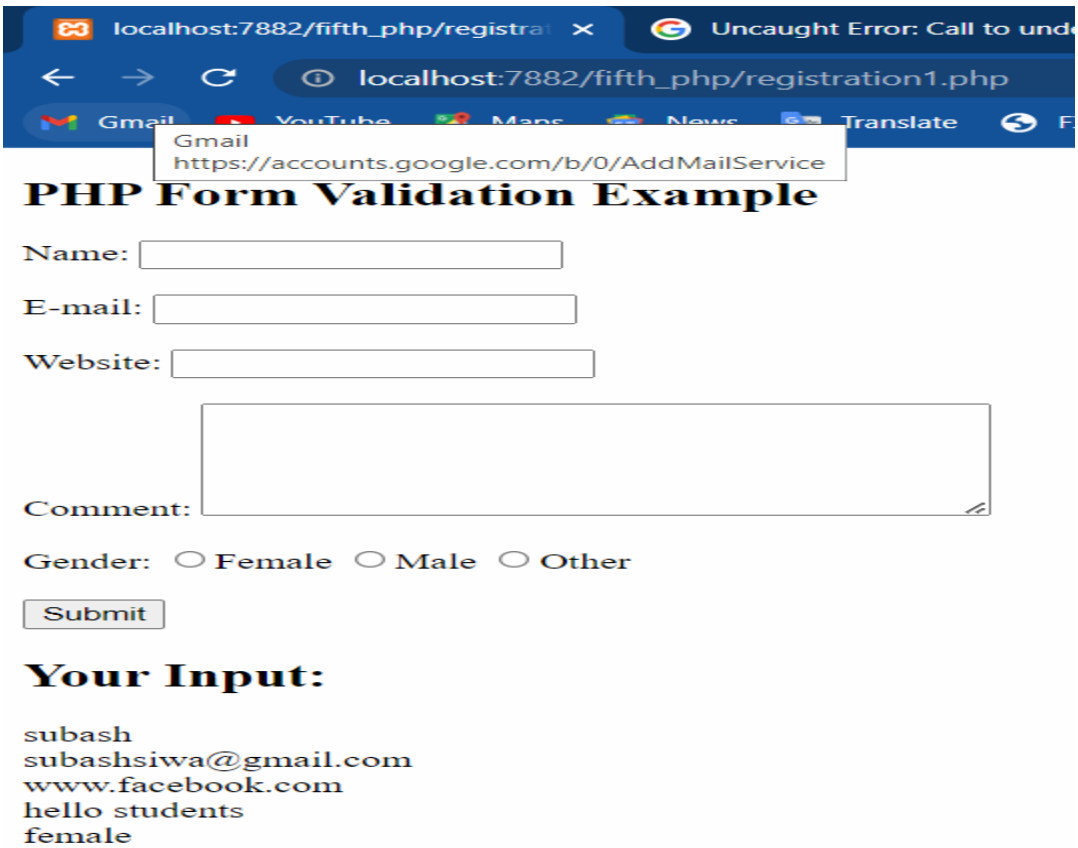


Example: 2

```
<!DOCTYPE HTML>
<html>
<head>
</head>
<body>

<?php
```

```php
// define variables and set to empty values
$name = $email = $gender = $comment = $website = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $name = test_input($_POST["name"]);
  $email = test_input($_POST["email"]);
  $website = test_input($_POST["website"]);
  $comment = test_input($_POST["comment"]);
  $gender = test_input($_POST["gender"]);
}

function test_input($data) {
  $data = trim($data);
  $data = stripslashes($data);
  $data = htmlspecialchars($data);
  return $data;
}
?>

<h2>PHP Form Validation Example</h2>
<form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">
  Name: <input type="text" name="name">
  <br><br>
  E-mail: <input type="text" name="email">
  <br><br>
  Website: <input type="text" name="website">
  <br><br>
  Comment: <textarea name="comment" rows="5" cols="40"></textarea>
  <br><br>
  Gender:
  <input type="radio" name="gender" value="female">Female
  <input type="radio" name="gender" value="male">Male
  <input type="radio" name="gender" value="other">Other
  <br><br>
  <input type="submit" name="submit" value="Submit">
</form>

<?php
echo "<h2>Your Input:</h2>";
echo $name;
echo "<br>";
echo $email;
echo "<br>";
echo $website;
echo "<br>";
```

```
echo $comment;
echo "<br>";
echo $gender;
?>

</body>
</html>
```



Setting Response Headers

- PHP and Apache normally take care of the headers for you (identify
  ing the document as HTML, calculating the length of the HTML page, etc.).
- Most web applications never need to set headers themselves. However, if you want to send back something that's not HTML, set the expiration time for a page, redirect the cli

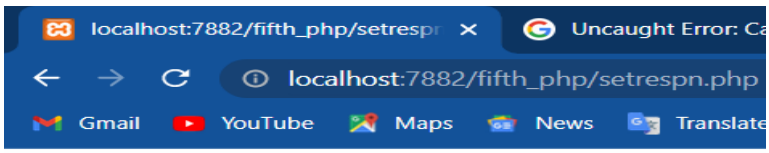
  ent's browser, or generate a specific HTTP error, you'll need to use the header()

function.

- The only catch to setting headers is that you must do so before any of the body is generated. This means that all calls to header() (or setcookie(), if you're setting cookies) must happen at the very top of your file, even before the <html> tag.

For example:

```
- <?php
- header('Content-Type:text/plain');
- ?>
- Date: today
- From: subash
- To: sushant
- Subject: well done!
- My son you are graet!!
```




```
Date: today
From: subash
 To: sushant
 Subject: well done!
 My son you are graet!!
```

1.Different Content Types

- The Content-Type header identifies the type of document being returned. Ordinarily this is "text/html", indicating an HTML document, but there are other useful docu ment types.
- For example, "text/plain" forces the browser to treat the page as plain text. This type is like an automatic "view source," and it is useful when debugging.

2.Redirections

- To send the browser to a new URL, known as a redirection, you set the Location header.
- Generally, you'll also exit immediately afterward, so the script doesn't bother generating and outputting the remainder of the code listing:

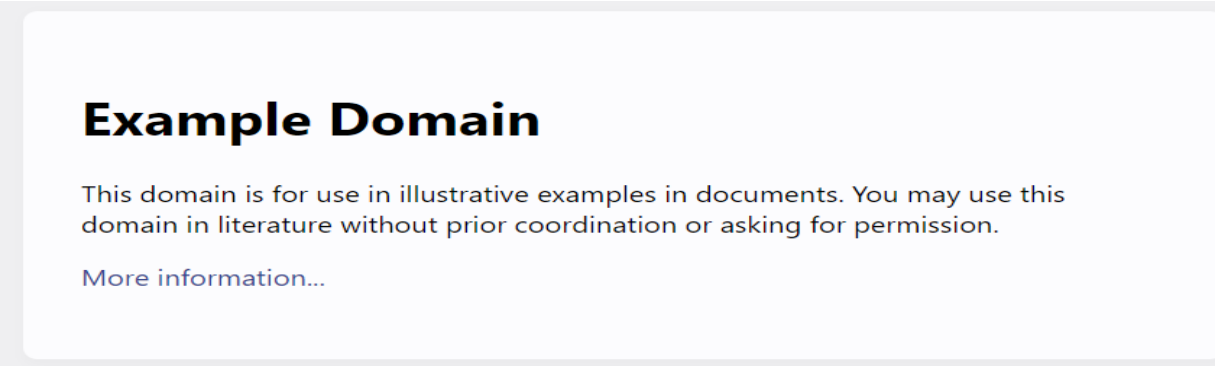
```
header("Location: http://www.example.com/elsewhere.html");
exit();
```

- When you provide a partial URL (e.g., /elsewhere.html), the web server handles this

redirection internally.

- This is only rarely useful, as the browser generally won't learn that it isn't getting the page it requested. If there are relative URLs in the new docu ment, the browser interprets those URLs as being relative to the requested document,rather than to the document that was ultimately sent. In general, you'll want to redi rect to an absolute URL.

```
<?php
header('Location: http://www.example.com/elsewhere.html');
exit( );
?>
```



3.Expiration

- A server can explicitly inform the browser, and any proxy caches that might be between the server and browser, of a specific date and time for the document to expire.
- Proxy and browser caches can hold the document until that time or expire it earlier.
- Repeated reloads of a cached document do not contact the server. However,an attempt to fetch an expired document does contact the server.

Here's an example of setting headers for a file download:

```
<?php
$file = 'example.zip';


header('Content-Description: File Transfer');
header('Content-Type: application/octet-stream');
header('Content-Disposition: attachment; filename="'.basename($file).'"');
header('Expires: 0');
header('Cache-Control: must-revalidate');
```

```
header('Pragma: public');
header('Content-Length: ' . filesize($file));
readfile($file);
?>
```

Today (1)

example	2/11/2023 5:22 PM	WinRAR ZIP archive	1 KB

A long time ago (213)

<u>To set the expiration time of a document, use the Expires header:</u>

header("Expires: Tue, 02 Jul 2019 05:30:00 GMT");

To force a document to expire three hours from the time the page was generated, use time() and gmstrftime() to generate the expiration date string:

$now = time();

$then = gmstrftime("%a, %d %b %Y %H:%M:%S GMT", $now + 60 * 60 * 3);

header("Expires: {$then}");

To indicate that a document "never" expires, use the time a year from now:

$now = time();

$then = gmstrftime("%a, %d %b %Y %H:%M:%S GMT", $now + 365 * 86440);

header("Expires: {$then}");

To mark a document as expired, use the current time or a time in the past:

$then = gmstrftime("%a, %d %b %Y %H:%M:%S GMT");

header("Expires: {$then}");

This is the best way to prevent a browser or proxy cache from storing your document:

header("Expires: Mon, 26 Jul 1997 05:00:00 GMT");

header("Last-Modified: " . gmdate("D, d M Y H:i:s") . " GMT");

header("Cache-Control: no-store, no-cache, must-revalidate");

header("Cache-Control: post-check=0, pre-check=0", false);

header("Pragma: no-cache");

<u>4.Authentication</u>

- HTTP authentication works through request headers and response statuses. A browser can send a username and password (the credentials) in the request headers.
- If the credentials aren't sent or aren't satisfactory, the server sends a "401 Unauthorized" response and identifies the realm of authentication (a string such as "Mary's Pictures" or "Your Shopping Cart") via the WWW-Authenticate header.

- This typi cally pops up an “Enter username and password for . . .” dialog box on the browser,and the page is then re-requested with the updated credentials in the header.
- To handle authentication in PHP, check the username and password (the PHP_AUTH_USER and PHP_AUTH_PW items of $_SERVER) and call header() to set the realm and send a “401 Unauthorized” response:

```
header('WWW-Authenticate: Basic realm="Top Secret Files"');
header("HTTP/1.0 401 Unauthorized");
```

You can do anything you want to authenticate the username and password; for exam ple, you could consult a database, read a file of valid users, or consult a Microsoft domain server.

This example checks to make sure that the password is the username reversed (not the most secure authentication method, to be sure!):

```
$authOK = false;
$user = $_SERVER['PHP_AUTH_USER'];
$password = $_SERVER['PHP_AUTH_PW'];
if (isset($user) && isset($password) && $user === strrev($password)) {
$authOK = true;
}
if (!$authOK) {
header('WWW-Authenticate: Basic realm="Top Secret Files"');
header('HTTP/1.0 401 Unauthorized');
// anything else printed here is only seen if the client hits "Cancel"
exit;
}
<!-- your password-protected document goes here -->
```

If you’re protecting more than one page, put the preceding code into a separate file and include it at the top of every protected page.

In PHP, you can set the HTTP response headers using the header() function. This function sends a raw HTTP header to the client and can be used to set the response status code, content type, and various other HTTP headers.

Here is an example of setting the content type to "text/html" and the response status code to "200 OK":

header("Content-Type: text/html");
header("HTTP/1.1 200 OK");
Note that the header() function must be called before any output is sent to the client, otherwise it will have no effect.

Here's an example of setting a custom header:

header("X-Custom-Header: MyValue");
Here's an example of setting the content type to "application/json" and sending a JSON response:

header("Content-Type: application/json");
$response = array("message" => "Hello, World!");
echo json_encode($response);
It is also possible to set multiple headers in one call by passing multiple arguments to the header() function:

header("Content-Type: text/html", true);
header("HTTP/1.1 200 OK", true);
In this example, the second argument true is used to indicate that the header should be appended to the list of headers, rather than replacing any existing headers with the same name.

Here's an example of setting response headers in PHP to send a JSON response:

```
<?php
header("Content-Type: application/json");
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: GET, POST");
header("Access-Control-Max-Age: 3600");
header("Access-Control-Allow-Headers: Content-Type, Access-Control-Allow-Headers,
Authorization, X-Requested-With");

$response = array("message" => "Hello, World!");
echo json_encode($response);
?>
```

In this example, we first set the Content-Type header to application/json to indicate that the response body contains JSON data. We then set four Access-Control-Allow-* headers to enable CORS (Cross-Origin Resource Sharing) and allow requests from any origin.

The Access-Control-Allow-Methods header specifies the allowed HTTP methods (GET and POST in this case), the Access-Control-Max-Age header specifies the maximum age of the CORS preflight response, and the Access-Control-Allow-Headers header specifies the allowed request headers.

Finally, we send a JSON response with a message "Hello, World!".

To create a PHP script that checks if a given number is odd or even using a self-processing form, follow these steps:

1. **Create the HTML form** within a PHP file.
2. **Process the form data** within the same PHP file upon form submission.

Here's an example of how to do this:

```
### Complete PHP Script (self_processing_form.php)
<!DOCTYPE html>
<html>
<head>
    <title>Odd or Even Checker</title>
</head>
<body>
    <h2>Check if a Number is Odd or Even</h2>

    <form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">
        <label for="number">Enter a number:</label>
        <input type="text" id="number" name="number"><br><br>
        <input type="submit" value="Check">
    </form>

    <?php
    if ($_SERVER["REQUEST_METHOD"] == "POST") {
        // Retrieve and sanitize the input
        $number = htmlspecialchars($_POST['number']);
```

```
        // Check if the input is a number
        if (is_numeric($number)) {
            // Check if the number is odd or even
            if ($number % 2 == 0) {
                echo "<p>The number $number is even.</p>";
            } else {
                echo "<p>The number $number is odd.</p>";
            }
        } else {
            echo "<p>Please enter a valid number.</p>";
        }
    }
    ?>
</body>
</html>
```

Explanation

1. **Form Creation**:
   - The form uses the `POST` method and the `action` attribute is set to `<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>`, which means the form will submit to itself.
   - An input field with `name="number"` allows the user to enter a number.
   - A submit button sends the form data to the server.

2. **Form Processing**:
   - When the form is submitted, `$_SERVER["REQUEST_METHOD"] == "POST"` checks if the form submission method is POST.
   - `htmlspecialchars($_POST['number'])` retrieves and sanitizes the input to prevent XSS attacks.
   - `is_numeric($number)` checks if the input is a valid number.
   - `$number % 2 == 0` checks if the number is even.

- Depending on the result, a message is displayed indicating whether the number is odd or even.

I apologize for any confusion. Let me clarify and ensure that the sticky form aspect of the form is correctly highlighted. The sticky form is achieved by setting the input field's value to the previously entered value after form submission. Here's the corrected script with a clear emphasis on the sticky form functionality:

Complete PHP Script with Sticky Form and Comments (sticky_form.php)

```
<!DOCTYPE html>
<html>
<head>
    <title>Odd or Even Checker</title>
</head>
<body>
    <h2>Check if a Number is Odd or Even</h2>

    <!-- Form for user input -->
    <form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">
        <label for="number">Enter a number:</label>
        <!-- Input field with sticky form functionality -->
                        <input type="text" id="number" name="number" value="<?php echo
isset($_POST['number']) ? htmlspecialchars($_POST['number']) : ''; ?>"><br><br>
        <input type="submit" value="Check">
    </form>

    <?php
    // Check if the form has been submitted
    if ($_SERVER["REQUEST_METHOD"] == "POST") {
        // Retrieve the input and sanitize it to prevent XSS attacks
        $number = htmlspecialchars($_POST['number']);

        // Check if the input is a numeric value
        if (is_numeric($number)) {
```

```
            // Determine if the number is even
            if ($number % 2 == 0) {
                echo "<p>The number $number is even.</p>"; // Output if the number is even
            } else {
                echo "<p>The number $number is odd.</p>"; // Output if the number is odd
            }
        } else {
            echo "<p>Please enter a valid number.</p>"; // Output if the input is not a number
        }
    }
    ?>
</body>
</html>
```

Explanation with Comments

1. **HTML Document Setup**:
   - `<!DOCTYPE html>`: Defines the document type and version of HTML.
   - `<html>`: Root element of the HTML document.
   - `<head>`: Contains metadata and title of the document.
   - `<title>Odd or Even Checker</title>`: Sets the title of the web page.
   - `<body>`: Contains the content of the HTML document.

2. **Form Creation**:
   - `<h2>Check if a Number is Odd or Even</h2>`: A heading for the form.
   - `<form method="post" action="<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>">`: Creates a form that uses the POST method to submit data to the same PHP script (self-processing form).
   - `<?php echo htmlspecialchars($_SERVER["PHP_SELF"]);?>`: Ensures the form submits to the same script, properly handling special characters to prevent XSS.
   - `<label for="number">Enter a number:</label>`: A label for the number input field.
   - `<input type="text" id="number" name="number" value="<?php echo isset($_POST['number']) ? htmlspecialchars($_POST['number']) : ''; ?>">`: An input field for

entering the number, with the value attribute set to retain the entered value after form submission.

- `<?php echo isset($_POST['number']) ? htmlspecialchars($_POST['number']) : ''; ?>`: Checks if the number was previously entered and sets it as the value of the input field.
- `<br><br>`: Line breaks for better form layout.
- `<input type="submit" value="Check">`: A submit button to send the form data.

3. **PHP Script for Form Processing**:
- `<?php`: Opens the PHP tag.
- `if ($_SERVER["REQUEST_METHOD"] == "POST")`: Checks if the form has been submitted using the POST method.
- `$number = htmlspecialchars($_POST['number']);`: Retrieves the input value and sanitizes it to prevent XSS attacks.
- `if (is_numeric($number))`: Checks if the input is a valid numeric value.
- `if ($number % 2 == 0)`: Checks if the number is even.
  - `echo "<p>The number $number is even.</p>";`: Outputs a message if the number is even.
- `else`: If the number is not even, it must be odd.
  - `echo "<p>The number $number is odd.</p>";`: Outputs a message if the number is odd.
- `else`: If the input is not a numeric value.
  - `echo "<p>Please enter a valid number.</p>";`: Outputs a message indicating the input is invalid.
- `?>`: Closes the PHP tag.

This script demonstrates how to create a sticky form in PHP to check if a given number is odd or even. The form retains the entered value after submission, providing a better user experience.

Example Usage
When a user enters a number and clicks the "Check" button:
- If the number is even, the output will be: `The number [number] is even.`
- If the number is odd, the output will be: `The number [number] is odd.`
- If the input is not a valid number, the output will be: `Please enter a valid number.`

This self-processing form in PHP effectively checks whether the entered number is odd or even and provides appropriate feedback.

To create a PHP script that checks if a given number is odd or even using a response header, we can use the `Location` header to redirect the user to another page or the same page with query parameters indicating the result.

Here’s how you can do it:

1. **Create a PHP file to handle form submission and redirect with results.**
2. **Create another PHP file to display the result based on query parameters.**

Step 1: Create the Form and Process Submission

```
<!-- form_handler.php -->
<!DOCTYPE html>
<html>
<head>
    <title>Odd or Even Checker</title>
</head>
<body>
    <h2>Check if a Number is Odd or Even</h2>

    <!-- Form for user input -->
    <form method="post" action="process_form.php">
        <label for="number">Enter a number:</label>
        <input type="text" id="number" name="number" value="<?php echo
isset($_POST['number']) ? htmlspecialchars($_POST['number']) : ''; ?>"><br><br>
        <input type="submit" value="Check">
    </form>
</body>
</html>
```

Step 2: Process the Form and Redirect

```
<!-- process_form.php -->
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Retrieve the input and sanitize it to prevent XSS attacks
    $number = htmlspecialchars($_POST['number']);

    // Check if the input is a numeric value
    if (is_numeric($number)) {
        // Determine if the number is even
```

```
        $result = ($number % 2 == 0) ? 'even' : 'odd';
    } else {
        $result = 'invalid';
    }

    // Redirect with result as a query parameter
    header("Location: result.php?number=" . urlencode($number) . "&result=" .
$result);
    exit(); // Ensure the script stops executing after the redirect
}
?>

### Step 3: Display the Result Based on Query Parameters

<!-- result.php -->
<!DOCTYPE html>
<html>
<head>
    <title>Odd or Even Checker Result</title>
</head>
<body>
    <h2>Result</h2>
    <?php
    if (isset($_GET['number']) && isset($_GET['result'])) {
        $number = htmlspecialchars($_GET['number']);
        $result = htmlspecialchars($_GET['result']);

        if ($result == 'even') {
            echo "<p>The number $number is even.</p>";
        } elseif ($result == 'odd') {
            echo "<p>The number $number is odd.</p>";
        } else {
            echo "<p>Please enter a valid number.</p>";
        }
    } else {
        echo "<p>No result to display. Please submit a number.</p>";
    }
    ?>
    <br>
    <a href="form_handler.php">Check Another Number</a>
</body>
</html>
```

Explanation

1. **form_handler.php**:
   - Displays a form for the user to enter a number.
   - The form submits to `process_form.php`.

2. **process_form.php**:
   - Checks if the form is submitted via POST.
   - Retrieves and sanitizes the input.
   - Checks if the input is numeric and determines if it is odd or even.
   - Redirects to `result.php` with the number and result as query parameters using the `Location` header.

3. **result.php**:
   - Retrieves the number and result from query parameters.
   - Displays whether the number is odd, even, or invalid.
   - Provides a link to go back to the form to check another number.

This method ensures that the form submission is processed, and the user is redirected with the result using response headers.